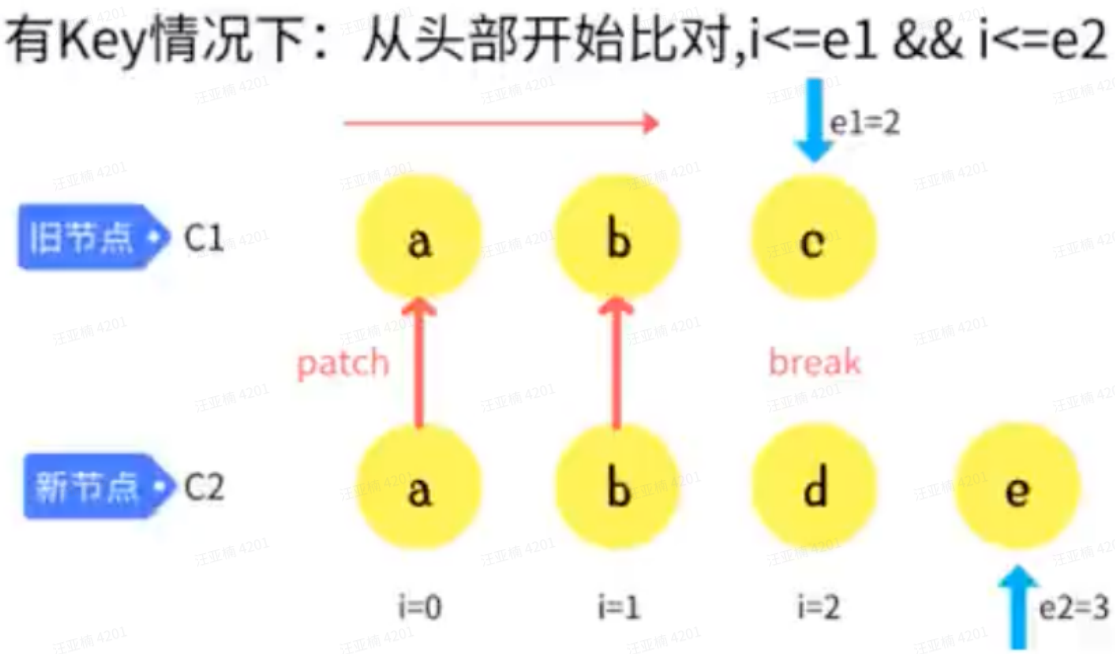
Vue3 Diff 算法

第一步

```
// old
h('div', { style: { color: 'red' } }, [
  h('li', { key: 'a' }, 'a'),
  h('li', { key: 'b' }, 'b'),
  h('li', { key: 'c' }, 'c'),
])

// new
h('div', { style: { color: 'blue' } }, [
  h('li', { key: 'a' }, 'a'),
  h('li', { key: 'b' }, 'b'),
  h('li', { key: 'd' }, 'd'),
  h('li', { key: 'e' }, 'e'),
])
```

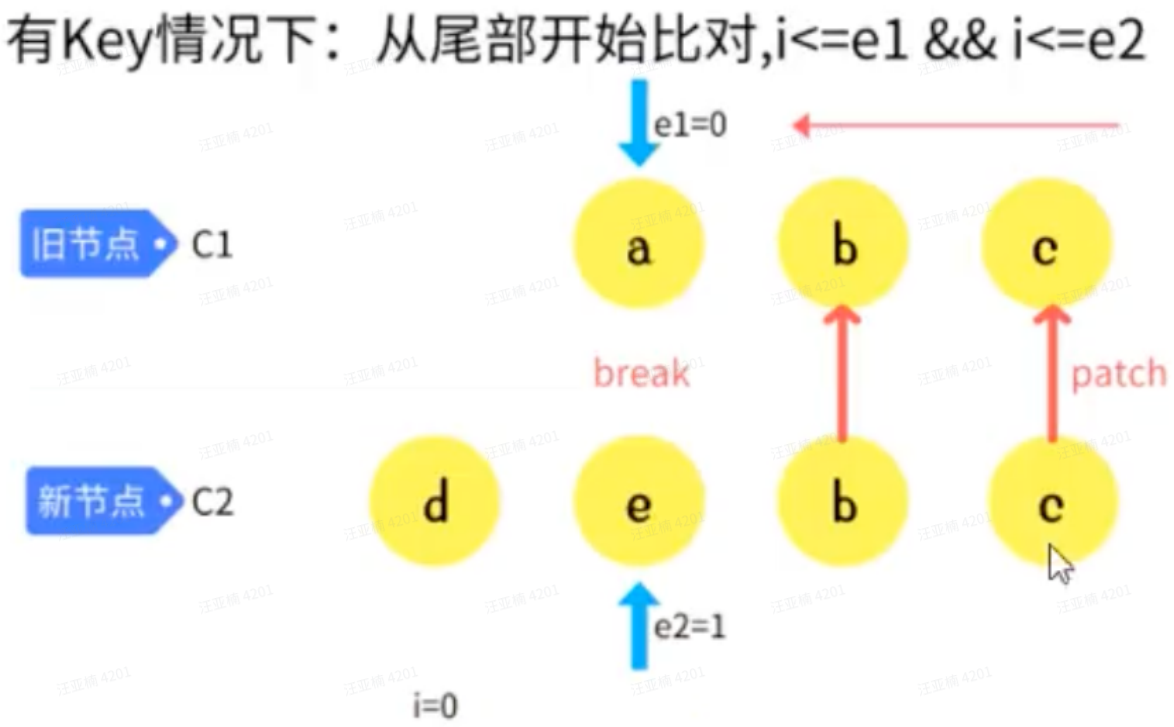



```
// sync from start: 从头开始一个个比 遇到不同的就停止
while (i <= e1 && i <= e2) {
```

```
  const n1 = c1[i]
  const n2 = c2[i]
  if (isSameVNodeType(n1, n2)) {
    patch(n1, n2, el)
  } else {
    break;
  }
  i++
}
```

第二步

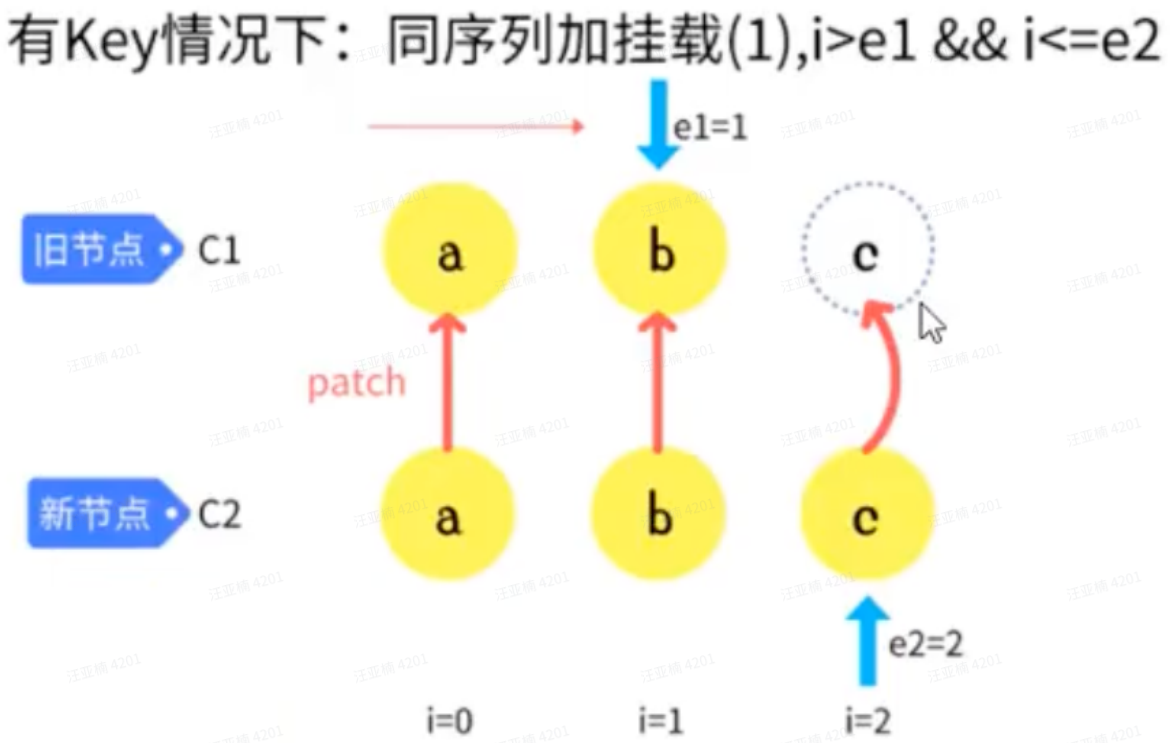
```
// old
h('div', { style: { color: 'red' } }, [
  h('li', { key: 'a' }, 'a'),
  h('li', { key: 'b' }, 'b'),
  h('li', { key: 'c' }, 'c'),
])

// new
h('div', { style: { color: 'blue' } }, [
  h('li', { key: 'd' }, 'd'),
  h('li', { key: 'e' }, 'e'),
  h('li', { key: 'b' }, 'b'),
  h('li', { key: 'c' }, 'c'),
])
```

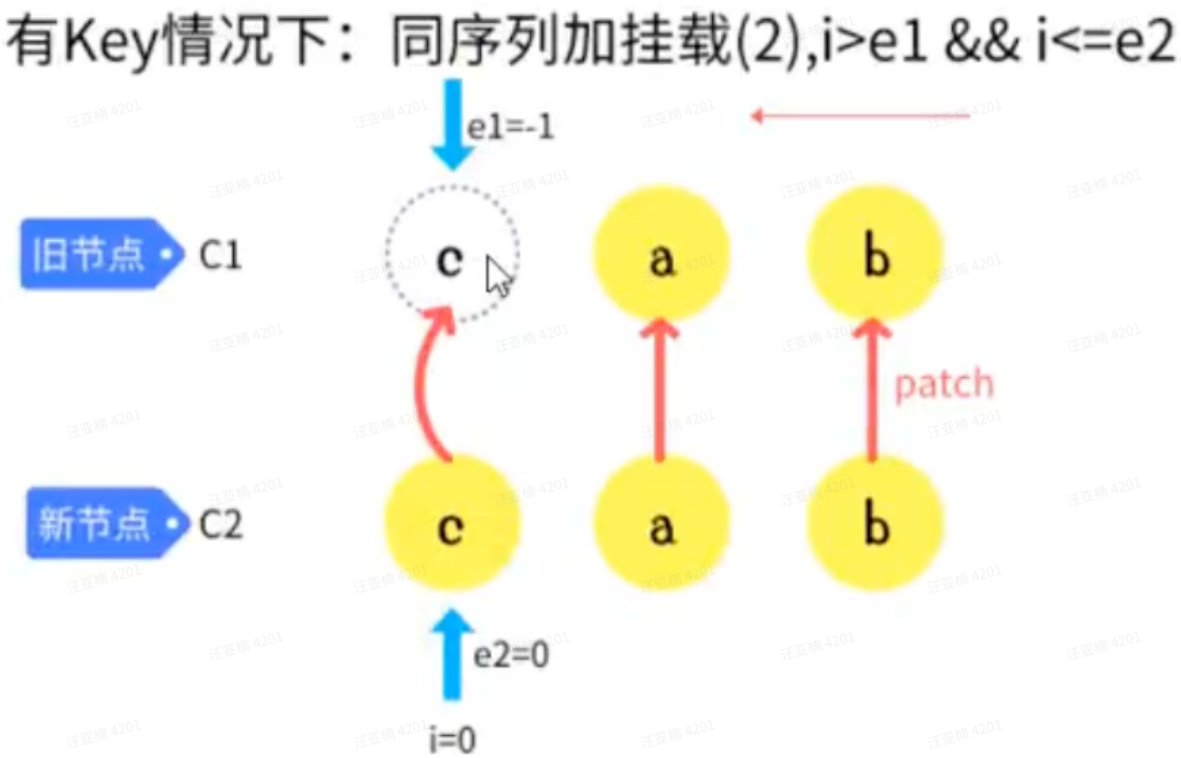



```
// sync from end: 从尾开始对比，遇到不同的就停止
while (i <= e1 && i <= e2) {
  const n1 = c1[e1]
  const n2 = c2[e2]
  if (isSameVNodeType(n1, n2)) {
    patch(n1, n2, el)
  } else {
    break;
  }
  e1--
  e2--
}
```

第三步: common sequence + mount

🍰 前提: 有一方已经对比完成



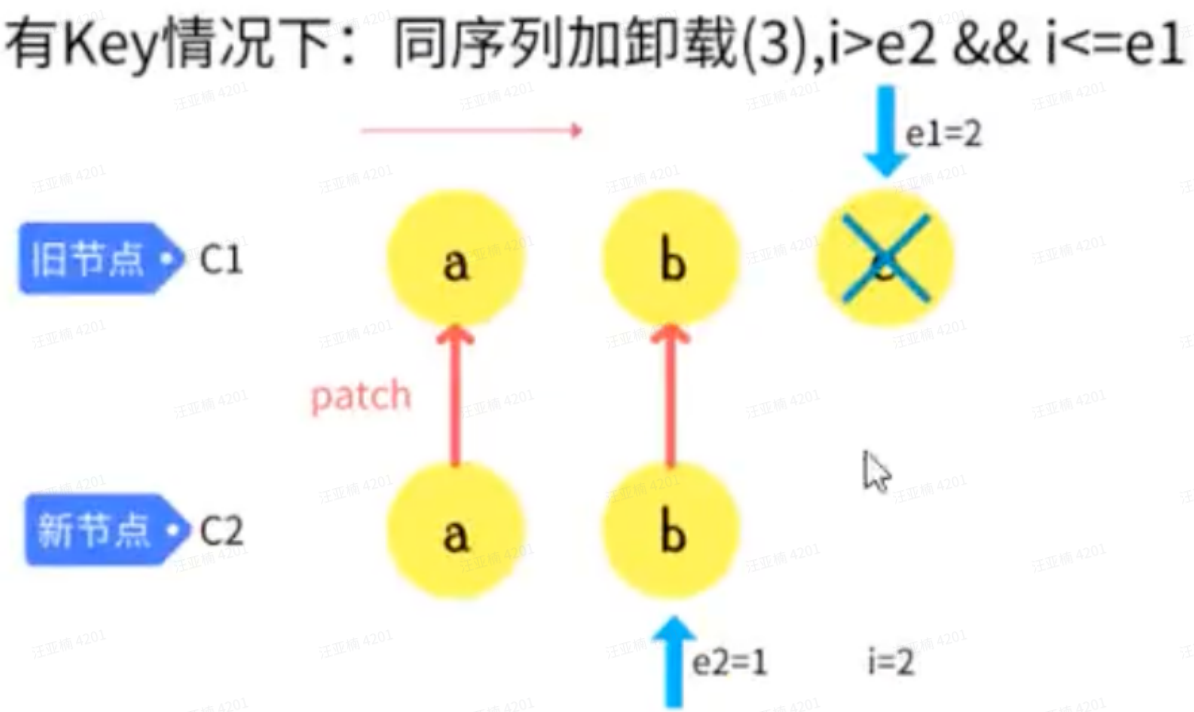


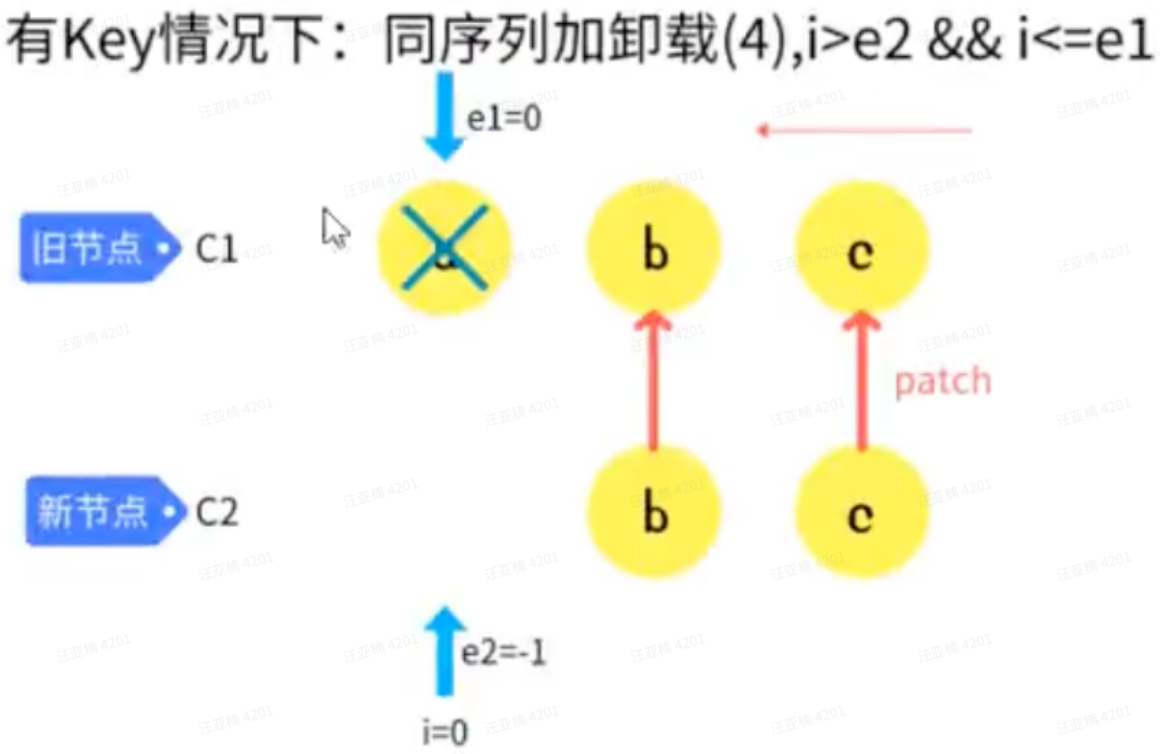

```
if (i > e1) { // common sequence + mount: 对比后 如有一方已经比对完全，老的少，新的多，去新增
  if (i <= e2) { // 表示有新增的部分
    const nextPos = e2 + 1
    // 想知道是向前插入 还是向右插入
    const anchor = nextPos < c2.length ? c2[nextPos].el : null
```

```
      while (i <= e2) {
        console.log('anchor: ', anchor)
        patch(null, c2[i], el, anchor)
        i++
      }
    }
  }
```

第四步: common sequence + unmount

> 🥛 前提: 有一方已经对比完成






```
else if (i > e2) { // common sequence + unmount: 对比后 如有一方已经比对完全，老的多，新的少 去卸载
  while (i <= e1) {
    unmount(c1[i])
    i++
  }
}
```

第五步: unknow sequence

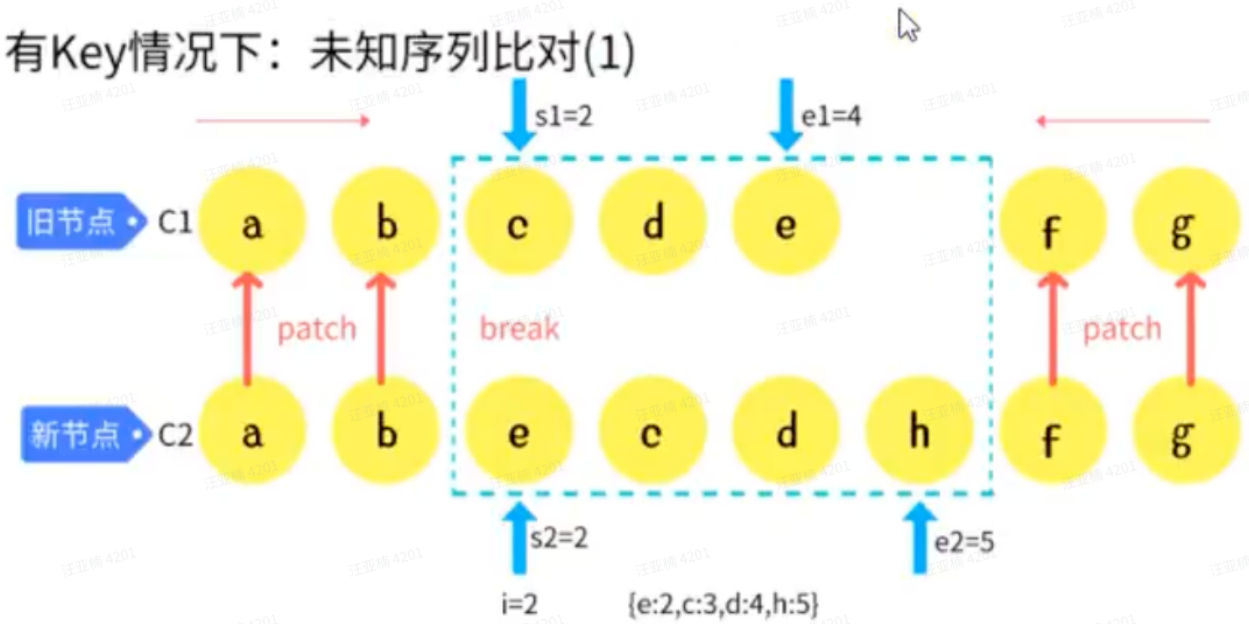
1. build key: index map for newChildren

建立映射表

```
// old
h('div', { style: { color: 'red' } }, [
  h('li', { key: 'a' }, 'a'),
  h('li', { key: 'b' }, 'b'),
  h('li', { key: 'c' }, 'c'),
  h('li', { key: 'd' }, 'd'),
  h('li', { key: 'e' }, 'e'),
  h('li', { key: 'f' }, 'f'),
  h('li', { key: 'g' }, 'g'),
```

```
10  ])
11
12  // new
13  h('div', { style: { color: 'blue' } }, [
14    h('li', { key: 'a' }, 'a'),
15    h('li', { key: 'b' }, 'b'),
16    h('li', { key: 'e' }, 'e'),
17    h('li', { key: 'c' }, 'c'),
18    h('li', { key: 'd' }, 'd'),
19    h('li', { key: 'h' }, 'h'),
20    h('li', { key: 'f' }, 'f'),
21    h('li', { key: 'g' }, 'g'),
22  ])
```

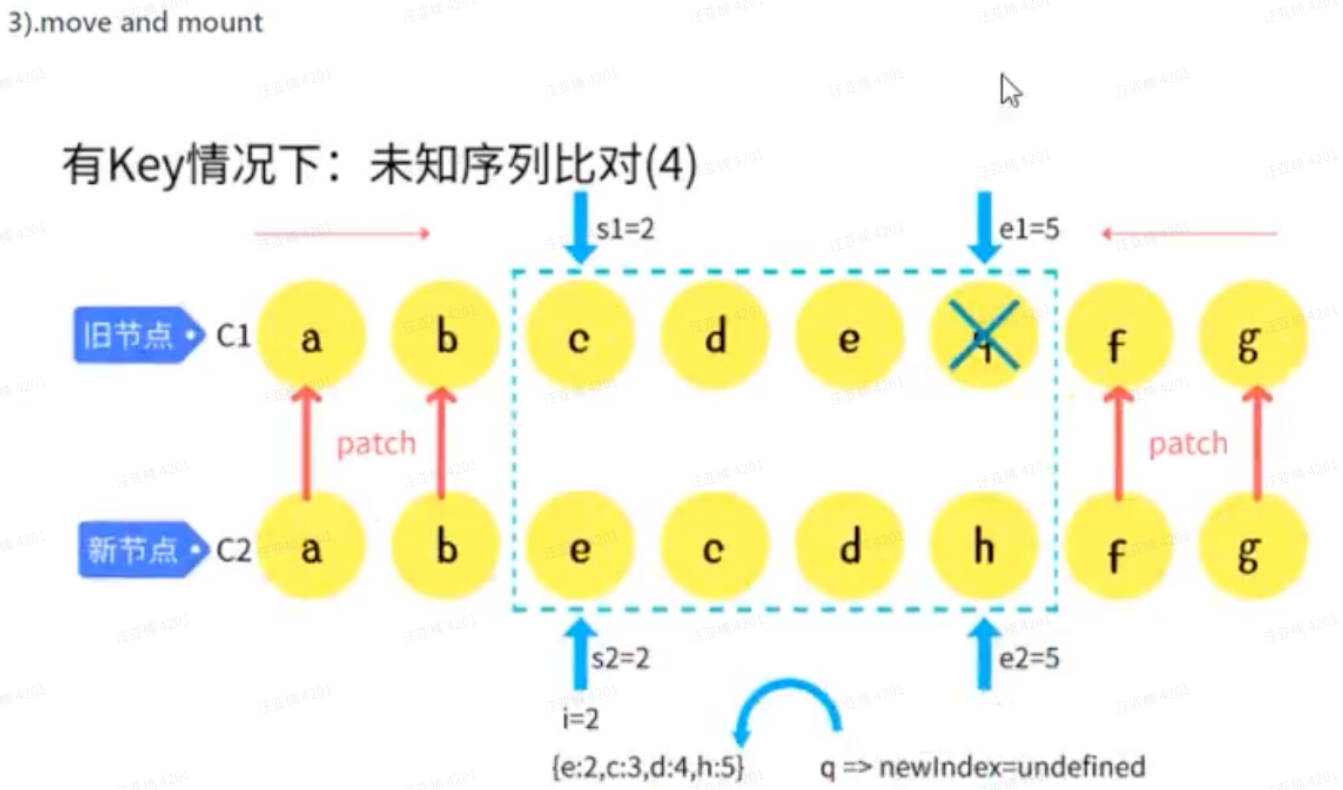
1). build key:index map for newChildren




```
1  let s1 = i
2  let s2 = i
3
4  // Vue3 用的是新的做映射表，Vue2 用的是老的做映射表
5  const keyToNewIndexMap = new Map()
6
7  for (let i = s2; i <= e2; i++) {
8    const childVNode = c2[i]
9    keyToNewIndexMap.set(childVNode.key, i)
10 }
```

2. loop through old children left to be patched and try to patch

循环遍历留下要修补的老孩子 并尝试修补

3. move and mount

移动以及挂载